

## Lab 04 haskell

### Ejercicio 1.

```
[13]: miLength :: [a] -> Int
      miLength [] = 0
      miLength (_:xs) = 1 + miLength xs
      miLength [2,4,6,8,10]
```

5

```
[12]: miLast :: [a] -> a
      miLast [x] = x
      miLast (_:xs) = miLast xs
      miLast [1,2,3,4,5,6]
```

6

```
[11]: duplicar :: [Int] -> [Int]
      duplicar xs = map (* 2) xs
      duplicar [1,12,3,4,5,6]
```

**Eta reduce**

**Found:**

**Why Not:**

```
duplicar xs = map (* 2) xs
duplicar = map (* 2)

[2,24,6,8,10,12]
```

```
eliminarPares :: [Int] -> [Int]
eliminarPares xs = filter odd xs
eliminarPares [1,2,3,4,5,6,7,8,9,10]
```

**Eta reduce**

**Found:**

**Why Not:**

```
eliminarPares xs = filter odd xs
eliminarPares = filter odd

[13,15,17,19]
```

```
miLength :: [a] -> Int
miLength xs = length xs
miLength [1..10]
```

### Eta reduce

**Found:**

**Why Not:**

```
miLength xs = length xsmiLength = length
10
```

```
miLast :: [a] -> a
miLast xs = last xs
miLast [1,3,6,7,8,3,4,5,6,8,0]
```

### Eta reduce

**Found:**

**Why Not:**

```
miLast xs = last xsmiLast = last
0
```

```
duplicar :: [Int] -> [Int]
duplicar = map(*2)
duplicar[10..20]
```

```
[20,22,24,26,28,30,32,34,36,38,40]
```

```
eliminarPares :: [Int] -> [Int]
eliminarPares = filter (\x -> mod x 2 /= 0)
eliminarPares [5,10..40]
```

### Use odd

**Found:**

**Why Not:**

```
mod x 2 /= 0odd x
[5,15,25,35]
```

---

## Ejercicio 2

1.

```
[x^2 | x <- [1..10]]
```

```
[1,4,9,16,25,36,49,64,81,100]
```

```
[x | x <- [1..30], mod x 3 == 0]
```

```
[3,6,9,12,15,18,21,24,27,30]
```

```
[ (x, y) | x <- [1..5], y <- [1..5], x < y]
```

```
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]
```

2.

```
divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n `mod` x == 0]
divisores 10
```

```
[1,2,5,10]
```

---

3.

```
esPerfecto :: Int -> Bool
esPerfecto n = sum [x | x <- [1..n-1], n `mod` x == 0] == n
esPerfecto 6
```

```
True
```

### Ejercicio 3

1.

a)

```
sumaLista :: [Int] -> Int
sumaLista xs = foldl (+) 0 xs
sumaLista [2,4,6,8,10]
```

#### Eta reduce

**Found:**

**Why Not:**

```
sumaLista xs = foldl (+) 0 xs
```

#### Use sum

**Found:**

**Why Not:**

```
foldl (+) 0 sum
```

30

b)

```
mayoresQue10 :: [Int] -> Int
mayoresQue10 xs = length(filter(\x -> x > 10)xs)
mayoresQue10 [1..20]
```

#### Avoid lambda using `infix`

**Found:**

**Why Not:**

```
(\ x -> x > 10)(> 10)
```

10

c)

```
productoLista :: [Int] -> Int
productoLista xs = foldl (*) 1 xs
productoLista [1..5]
```

#### Eta reduce

##### Found:

##### Why Not:

```
productoLista xs = foldl (*) 1 xs
productoLista = foldl (*) 1
```

#### Use product

##### Found:

##### Why Not:

```
foldl (*) 1 product
```

120

d)

```
celsiusAFahrenheit :: [Double] -> [Double]
celsiusAFahrenheit xs = map (\x -> x * 9 / 5 + 32) xs
celsiusAFahrenheit [1,20,30]
```

#### Eta reduce

##### Found:

##### Why Not:

```
celsiusAFahrenheit xs = map (\x -> x * 9 / 5 + 32) xs
celsiusAFahrenheit = map (\x -> x * 9 / 5 + 32)

[33.8,68.0,86.0]
```

e)

```
ajustarImpares :: [Int] -> [Int]
ajustarImpares xs = map (\x -> if mod x 2 /= 0 then x + 5 else x) xs
ajustarImpares [1..10]
```

#### Eta reduce

##### Found:

##### Why Not:

```
ajustarImpares xs = map (\x -> if mod x 2 /= 0 then x + 5 else x) xs
ajustarImpares = map (\x -> if mod x 2 /= 0 then x + 5 else x)
```

#### Use odd

##### Found:

##### Why Not:

```
mod x 2 /= 0 odd x

[6,2,8,4,10,6,12,8,14,10]
```

```
ajustarImpares :: [Int] -> [Int]
ajustarImpares xs = map (\x -> if odd x then x + 5 else x) xs
ajustarImpares [1..10]
```

#### Eta reduce

##### Found:

##### Why Not:

```
ajustarImpares xs = map (\x -> if odd x then x + 5 else x) xs
ajustarImpares = map (\x -> if odd x then x + 5 else x)

[6,2,8,4,10,6,12,8,14,10]
```

2.

a)

```
nombresLargos :: [String] -> [String]
nombresLargos xs = filter (\s -> length s > 5) xs
nombresLargos ["Julian", "Ezequiel", "Lorenzo", "Dario"]
```

#### Eta reduce

##### Found:

##### Why Not:

```
nombresLargos xs = filter (\s -> length s > 5) xs
nombresLargos = filter (\s -> length s > 5)

["Julian", "Ezequiel", "Lorenzo"]
```

b)

```
soloNegativos :: [Int] -> [Int]
soloNegativos xs = filter(\s -> s < 0)xs
soloNegativos [-4..4]
```

**Eta reduce**

**Found:**

**Why Not:**

```
soloNegativos xs = filter (\ s -> s < 0) xs
```

**Avoid lambda using `infix`**

**Found:**

**Why Not:**

```
(\ s -> s < 0)(< 0)
```

```
[-4,-3,-2,-1]
```

c)

```
edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios xs = filter(\s -> s >= 18 && s <= 25)xs
edadesUniversitarios [10..30]
```

**Eta reduce**

**Found:**

```
edadesUniversitarios xs = filter (\ s -> s >= 18 && s <= 25)
[18,19,20,21,22,23,24,25]
```

3.

a)

```
: agregarIVA :: [Double] -> [Double]
agregarIVA xs = map (\x -> x + (x * 0.19)) (filter (\s -> s >= 1000) xs)
agregarIVA [995..1005]
```

**Avoid lambda using `infix`**

**Found:**

**Why Not:**

```
(\ s -> s >= 1000)(>= 1000)
```

```
[1190.0,1191.19,1192.38,1193.57,1194.76,1195.95]
```

b)

```
longitudesVocales :: [String] -> [Int]
longitudesVocales = map length . filter (\w -> not (null w) && head w `elem` "AEIOUaeiou")
longitudesVocales ["arbol","gato","Arroz","Eureka"]

[5,5,6]
```

c)

```
cuadradosPares :: [Int] -> [Int]
cuadradosPares xs = map(\x -> x*2) (filter(\x -> even x) xs)
cuadradosPares [1..10]
```

**Avoid lambda using `infix`**

**Found:**                      **Why Not:**

```
(\ x -> x * 2)(* 2)
```

**Avoid lambda**

**Found:**                      **Why Not:**

```
(\ x -> even x)even
```

```
[4,8,12,16,20]
```

4.

a)

```
4]: hayNegativo :: [Int] -> Bool
hayNegativo xs = any(\x -> x < 0) xs
hayNegativo[-5..5]
```

**Eta reduce**

**Found:**                                      **Why Not:**

```
hayNegativo xs = any (\ x -> x < 0) xs hayNegativo = any (\ x -> x < 0)
```

**Avoid lambda using `infix`**

**Found:**                      **Why Not:**

```
(\ x -> x < 0)(< 0)
```

```
True
```

---

```
1]: hayNegativo :: [Int] -> Bool
hayNegativo = any (< 0)
hayNegativo[-5..5]
```

```
True
```

b)

```
todosPares :: [Int] -> Bool
todosPares xs = all even xs
todosPares [2,4..10]
```

### Eta reduce

**Found:**

**Why Not:**

```
todosPares xs = all even xstodosPares = all even
```

True

---

c)

```
import Data.Char (isUpper)
nombresCorrectos :: [String] -> Bool
nombresCorrectos xs = all (\x -> isUpper(head x)) xs
nombresCorrectos ["Andres","Miguel", "Sebastian", "Luis"]
```

### Eta reduce

**Found:**

**Why Not:**

```
nombresCorrectos xs = all (\ x -> isUpper (head x)) xsnombresCorrectos = all (\ x -> isUpper (head x))
```

### Avoid lambda

**Found:**

**Why Not:**

```
\ x -> isUpper (head x)isUpper . head
```

True

5.

a)

```
menoresQue100 :: [Int] -> [Int]
menoresQue100 = takeWhile (\x -> x < 100)
menoresQue100 [90,92,110]
```

### Avoid lambda using `infix`

**Found:**

**Why Not:**

```
(\ x -> x < 100)(< 100)
```

[90,92]

b)

```

sinCerosIniciales :: [Int] -> [Int]
sinCerosIniciales xs = dropWhile(==0) xs
sinCerosIniciales [0,3,5,5]
sinCerosIniciales [0,0,0,0,0,0,1]

```

#### Eta reduce

##### Found:

```

sinCerosIniciales xs = dropWhile (== 0) xs
sinCerosIniciales [3,5,5]
sinCerosIniciales [1]

```

##### Why Not:

## Ejercicio 4

```

]: procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
   procesarLecturas f xs = map f xs

   celsiusAFahrenheit :: Double -> Double
   celsiusAFahrenheit c = c * 9 / 5 + 32

   crearFiltro :: (a -> Bool) -> ([a] -> [a])
   crearFiltro p = filter p

   soloPares :: [Int] -> [Int]
   soloPares = crearFiltro even

   enRango :: Int -> Int -> (Int -> Bool)
   enRango minimo maximo = \x -> x >= minimo && x <= maximo

   entre5y10 :: Int -> Bool
   entre5y10 = enRango 5 10

   potenciador :: Int -> (Int -> Int)
   potenciador n = \x -> x ^ n

   potencia5 :: Int -> Int
   potencia5 = potenciador 5

```

Eta reduce

```
[22]: procesarLecturas celsiusAFaranheit [0,10,25]  
      [32.0,50.0,77.0]
```

```
[23]: soloPares[1..20]  
      [2,4,6,8,10,12,14,16,18,20]
```

```
[25]: entre5y10 7  
      True
```

```
[26]: potencia5 7  
      16807
```

```
[ ]:
```

## Ejercicio 5

```
[1]: import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo   = Estado
type Costo  = Int
type Grafo  = [(Nodo, [(Nodo, Costo)])]

capA, capB :: Int
capA = 5
capB = 3

[2]: -- Verter de A a B: se vierte lo que quepa en B o lo que tenga A
transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - t, b + t)
  where t = min a (capB - b)

-- Verter de B a A
transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + t, b - t)
  where t = min b (capA - a)
```

```
[4]: vecinos :: Estado -> [(Estado, Costo)]
vecinos (a, b) =
  [ ((capA, b), 1)      -- Llenar A
  , ((a, capB), 1)      -- Llenar B
  , ((0, b), 1)         -- Vaciar A
  , ((a, 0), 1)         -- Vaciar B
  , (transferirAB (a, b), 1) -- Verter A->B
  , (transferirBA (a, b), 1) -- Verter B->A
  ]

-- Admisible: si no estamos en la meta hace falta al menos 1 acción
heuristica :: Estado -> Costo
heuristica (4, 0) = 0
heuristica _      = 1
```

```
[5]: aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heuristica inicio meta = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == meta = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
            ordenados = sortOn (\(v, c) -> c + heuristica v) (cola ++ nuevos)
        in (nodo, costo) : buscar ordenados (nodo : visitados)

aStar vecinos heuristica (0, 0) (4, 0)
[ ((0,0),0),((5,0),1),((0,3),1),((5,3),2),((2,3),2),((3,0),2),((2,0),3),((3,3),3),((0,2),4),((5,1),4),((5,2),5),((0,1),5),((4,3),6),((1,0),6),((4,0),7)]
```

```
[6]: vecinosAcc :: Estado -> [(String, Estado)]
vecinosAcc e@(a, b) =
  [ ("Llenar A", (capA, b))
  , ("Llenar B", (a, capB))
  , ("Vaciar A", (0, b))
  , ("Vaciar B", (a, 0))
  , ("Verter A->B", transferirAB e)
  , ("Verter B->A", transferirBA e)
  ]

aStarCamino :: Estado -> Estado -> Maybe (Costo, [(String, Estado)])
aStarCamino inicio meta = buscar [(heuristica inicio, 0, inicio, [])] []
  where
    buscar [] _ = Nothing
    buscar ((_, g, e, camino):cola) visitados
      | e == meta = Just (g, reverse camino)
      | e `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (g + 1 + heuristica v, g + 1, v, (acc, v) : camino)
                      | (acc, v) <- vecinosAcc e, v `notElem` visitados ]
            cola' = sortOn (\(f, _, _) -> f) (cola ++ nuevos)
        in buscar cola' (e : visitados)
```

```
[7]: case aStarCamino (0, 0) (4, 0) of
      Nothing -> putStrLn "No hay solución"
      Just (costo, pasos) -> do
        putStrLn ("Costo total: " ++ show costo)
        putStrLn "(0,0) <- estado inicial"
        mapM_ (\(acc, est) -> putStrLn (show est ++ " <- " ++ acc)) pasos
```

```
Costo total: 7
(0,0) <- estado inicial
(5,0) <- Llenar A
(2,3) <- Verter A->B
(2,0) <- Vaciar B
(0,2) <- Verter A->B
(5,2) <- Llenar A
(4,3) <- Verter A->B
(4,0) <- Vaciar B
```

## Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?  
(Horas/Hombre)  
David Cardenas: 5 horas  
Lorenzo Baez: 5 horas
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué? Culminado, en clase se alcanzaron a terminar 3 de los 5 ejercicios propuestos, se terminó gracias a la adecuada organización del equipo.

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El completar el laboratorio, ya que hubo algunos ejercicios que tomaron bastante tiempo para desarrollarlos.

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

No se presentaron problemas técnicos, todo se pudo resolver con normalidad, gracias a la comunicación dentro del equipo.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

La distribución de los ejercicios, y continuar con el buen manejo del tiempo y procurar acabar los laboratorios en clase.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

El material proporcionado en Moodle.

<http://aprendehaskell.es/main.html>